P.Vishwa Nandan(192473015)

Q7 – Diagnose, Redesign and Validate

Marks: 13

Selected Solution: Q5 – Generic Leaderboard

1. Introduction

The original Leaderboard application ranks sports teams based on points, goal difference and team name.

It uses a generic class and Comparator to arrange teams in the correct ranking order.

However, the original implementation creates a new list and sorts all the elements every time the ranking() method is called.

When the number of teams increases and the ranking is requested repeatedly, unnecessary sorting operations and object creation occur.

Therefore, the original design is modified using a caching mechanism to avoid repeated sorting.

A. Diagnose the Bottleneck (2 Marks)

2. Identifying the Problem

The original Leaderboard uses the following method:

```
List<T> ranking() {

    List<T> sortedItems = new ArrayList<>(items);

    sortedItems.sort(order);

    return sortedItems;
}
```

Every time this method is called:

1. A new ArrayList is created.
2. All team references are copied into the new list.
3. The complete list is sorted using Comparator.
4. The sorted list is returned.

Even when no new teams are added, the same sorting operation is repeated.

3. Performance Limitation

Let n represent the number of teams.

The original implementation has the following complexity:

- Copying elements: O(n).
- Sorting elements: O(n log n).
- Total time: O(n log n).
- Additional temporary space: O(n).

If the ranking method is called 10 times without adding new teams, the same list is copied and sorted 10 times.

This creates unnecessary work.

4. Bottleneck Analysis

The main bottleneck is repeated sorting and unnecessary ArrayList creation.

The following table shows the number of sorting operations for repeated ranking requests without any new team additions.

Number of Teams	Ranking Requests	Sorting Operations
1,000	10	10
10,000	10	10
50,000	10	10

The sorting operation becomes more expensive as the number of teams increases.

Therefore, the original design is not efficient for applications where rankings are requested frequently but the underlying team data changes less often.

B. Define the Design Objective (2 Marks)

5. Objectives of the Redesigned System

The redesigned system must achieve the following objectives:

1. Avoid sorting the leaderboard repeatedly when no new teams are added.
2. Reduce unnecessary ArrayList creation.
3. Maintain the correct ranking order.
4. Update the ranking whenever a new team is added.

5. Preserve the original ranking rules.
6. Improve performance when the leaderboard is accessed repeatedly.

6. Measurable Objective

The main objective is to reduce the number of complete sorting operations.

For example, when the ranking is requested 10 times without any changes:

Original Design:
10 ranking requests = 10 sorting operations.

Redesigned Design:
10 ranking requests = 1 sorting operation.

This reduces the number of sorting operations by 90%.

The improved implementation should also reduce repeated copying of team references.

C. Design the Revised Solution (4 Marks)

7. Revised Design – Cached Leaderboard

The redesigned solution uses a caching mechanism.

Caching means storing an already calculated result and reusing it when the same result is requested again.

Instead of sorting every time, the system sorts the teams only when necessary.

Working Process

1. When a team is added, the existing ranking cache is invalidated.
2. When ranking() is called for the first time, the teams are copied and sorted.
3. The sorted result is stored in a cache.
4. When ranking() is called again without any new additions, the cached result is returned directly.
5. When another team is added, the cache is invalidated and rebuilt during the next ranking request.

8. Revised Class Structure

The redesigned system contains the following components:

Team Class:

Stores team name, points and goal difference.

Leaderboard Class:

Maintains the original list of teams and calculates the ranking.

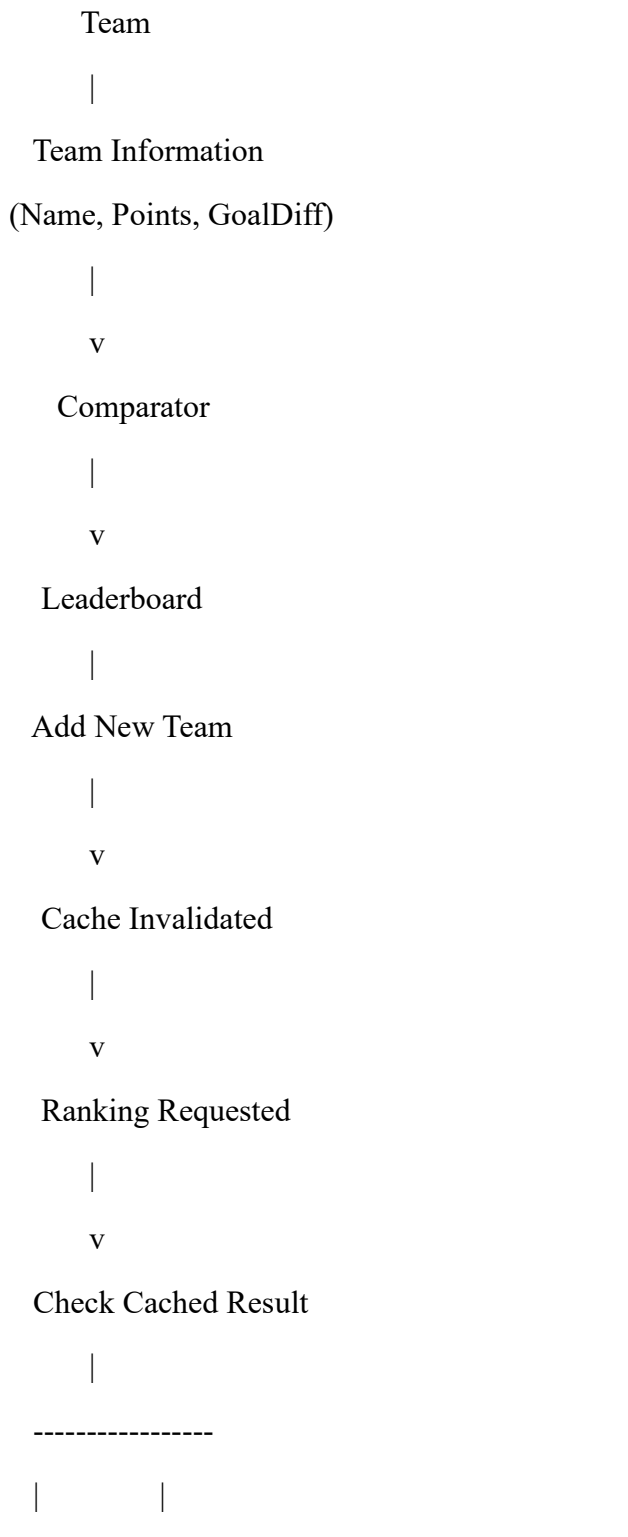
CachedLeaderboard Class:

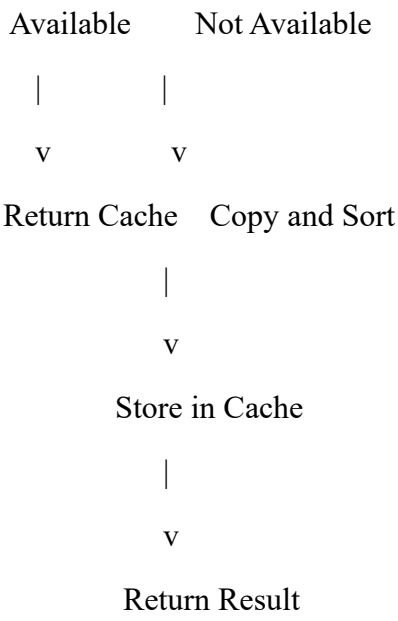
Maintains a sorted ranking and reuses it until the team data changes.

Comparator:

Defines the ranking order based on points, goal difference and team name.

Design Representation





9. Revised Algorithm

Algorithm: Cached Ranking

```
BEGIN

   ADD team to leaderboard

   INVALIDATE cached ranking

END



ALGORITHM Ranking()

BEGIN

   IF cached ranking is not available THEN

      CREATE a copy of the team list
```

SORT the copied list using Comparator

STORE the sorted list in cache

END IF

RETURN cached ranking

END

10. Java Implementation – Original and Redesigned Solutions

File name: Main.java

The following program compares the original Leaderboard implementation with the redesigned CachedLeaderboard implementation.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Team {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {

        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
```

```
    }

    @Override
    public String toString() {

        return name + "(" + points + "," + goalDiff + ")";
    }
}

// ORIGINAL LEADERBOARD
class Leaderboard<T> {

    private final List<T> items = new ArrayList<>();

    private final Comparator<? super T> order;

    private int sortCount = 0;

    Leaderboard(Comparator<? super T> order) {

        this.order = order;
    }

    void add(T item) {

        items.add(item);
    }

    // Original method sorts every time
```

```
    List<T> ranking() {

        List<T> sortedItems = new ArrayList<>(items);

        sortedItems.sort(order);

        sortCount++;

        return sortedItems;
    }

    int getSortCount() {

        return sortCount;
    }
}

// REDESIGNED CACHED LEADERBOARD
class CachedLeaderboard<T> {

    private final List<T> items = new ArrayList<>();

    private final Comparator<? super T> order;

    private List<T> cachedRanking = null;

    private List<T> cachedView = null;

    private int sortCount = 0;
```

```
CachedLeaderboard(Comparator<? super T> order) {

    this.order = order;
}

// Add team and invalidate the cache
void add(T item) {

    items.add(item);

    cachedRanking = null;

    cachedView = null;
}

// Sort only when the cache is unavailable
List<T> ranking() {

    if (cachedRanking == null) {

        cachedRanking = new ArrayList<>(items);

        cachedRanking.sort(order);

        cachedView = Collections.unmodifiableList(
                cachedRanking
        );
```

```
            sortCount++;
        }

        return cachedView;
    }

    int getSortCount() {

        return sortCount;
    }
}

// Main class
public class Main {

    static volatile int result;

    // Generate teams for testing
    static List<Team> createTeams(int n) {

        List<Team> teams = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            String name = "Team" + i;

            int points = (i * 37) % 101;

            int goalDiff = (i * 17) % 51;
```

```
            teams.add(
                new Team(name, points, goalDiff)
            );
        }

        return teams;
    }

    // Test correctness of both implementations
    static void correctnessTest(
            Comparator<Team> order) {

        Leaderboard<Team> original =
                new Leaderboard<>(order);

        CachedLeaderboard<Team> redesigned =
                new CachedLeaderboard<>(order);

        List<Team> teams = new ArrayList<>();

        teams.add(new Team("Panthers", 9, 4));

        teams.add(new Team("Eagles", 9, 6));

        teams.add(new Team("Cobras", 6, 1));

        teams.add(new Team("Bisons", 9, 4));
```

```
    for (Team team : teams) {

        original.add(team);

        redesigned.add(team);
    }

    System.out.println("CORRECTNESS TEST");

    System.out.println(
            "Original Ranking: " + original.ranking()
    );

    System.out.println(
            "Redesigned Ranking: " + redesigned.ranking()
    );
}

// Measure original solution
static long measureOriginal(
        List<Team> teams,
        Comparator<Team> order,
        int requests) {

    Leaderboard<Team> board =
            new Leaderboard<>(order);

    for (Team team : teams) {
        board.add(team);
```

```
    }

    long start = System.nanoTime();

    for (int i = 0; i < requests; i++) {

        List<Team> ranking = board.ranking();

        result += ranking.get(0).points;
    }

    long end = System.nanoTime();

    System.out.println(
            "Original sorting operations: "
            + board.getSortCount()
    );

    return end - start;
}

// Measure redesigned solution
static long measureRedesigned(
        List<Team> teams,
        Comparator<Team> order,
        int requests) {

    CachedLeaderboard<Team> board =
            new CachedLeaderboard<>(order);
```

```
    for (Team team : teams) {
        board.add(team);
    }

    long start = System.nanoTime();

    for (int i = 0; i < requests; i++) {

        List<Team> ranking = board.ranking();

        result += ranking.get(0).points;
    }

    long end = System.nanoTime();

    System.out.println(
            "Redesigned sorting operations: "
            + board.getSortCount()
    );

    return end - start;
}

// Compare original and redesigned systems
static void test(int n, Comparator<Team> order) {

    int requests = 10;
```

```
List<Team> teams = createTeams(n);

System.out.println("\n================================");

System.out.println("NUMBER OF TEAMS: " + n);

System.out.println("RANKING REQUESTS: " + requests);

System.out.println("================================");

long originalTime =
        measureOriginal(teams, order, requests);

long redesignedTime =
        measureRedesigned(teams, order, requests);

System.out.printf(
        "Original Time: %.3f ms%n",
        originalTime / 1_000_000.0
);

System.out.printf(
        "Redesigned Time: %.3f ms%n",
        redesignedTime / 1_000_000.0
);

System.out.println(
        "Original copied references: "
        + (long) n * requests
```

```
    );

    System.out.println(
            "Redesigned copied references: "
            + n
    );
}

public static void main(String[] args) {

    Comparator<Team> byRank =
            Comparator.comparingInt(
                    (Team t) -> t.points
            ).reversed()

            .thenComparing(
                    Comparator.comparingInt(
                            (Team t) -> t.goalDiff
                    ).reversed()
            )

            .thenComparing(t -> t.name);

    correctnessTest(byRank);

    test(1000, byRank);

    test(10000, byRank);
```

```
        test(50000, byRank);

        System.out.println("\nRedesign testing completed.");
    }
}
```

11. Explanation of the Redesigned Code

A. Cache Invalidation

```
void add(T item) {

    items.add(item);

    cachedRanking = null;

    cachedView = null;
}
```

Whenever a new team is added, the old ranking becomes outdated.

Therefore, both cache references are set to null.

B. Checking the Cache

```
if (cachedRanking == null) {
```

This checks whether a valid ranking is already available.

If the cache is available, sorting is not performed again.

C. Sorting and Storing

```
cachedRanking = new ArrayList<>(items);

cachedRanking.sort(order);
```

A new copy of the original list is created and sorted.

The sorted result is stored in the cache.

D. Returning the Cached Result

```
return cachedView;
```

The previously calculated ranking is returned directly.

Collections.unmodifiableList() prevents the calling code from modifying the cached result.

This maintains the correctness of the cached ranking.

12. Advantages of the Revised Solution

1. Avoids repeated sorting operations.
2. Reduces unnecessary ArrayList creation.
3. Improves performance when rankings are requested repeatedly.
4. Maintains the original ranking order.
5. Automatically invalidates the cache when a new team is added.
6. Makes the leaderboard more efficient for read-heavy applications.

13. Trade-offs

The redesigned solution also has some limitations.

1. The cached ranking requires additional memory.
2. When a new team is added, the next ranking request must sort the complete list again.
3. If teams are added frequently and rankings are requested after every addition, the improvement may be small.
4. The ranking fields should remain unchanged while the team is stored, or the cache may become incorrect. The Team class uses final fields to avoid this problem.

D. Predict Before Implementing (1 Mark)

14. Predicted Behaviour

Before executing the redesigned program, the following improvements are predicted.

Original Solution:

- Sorts the complete team list every time ranking is requested.
- Creates a new ArrayList for every ranking request.
- Performs repeated sorting and copying.

Redesigned Solution:

- Sorts the team list only when the cache is unavailable.
- Reuses the cached ranking for subsequent requests.
- Invalidates the cache only when a new team is added.

Predicted Comparison

For 10 ranking requests without adding new teams:

Measurement	Original	Redesigned
Sorting operations	10	1
Ranking list copies	10	1
References copied for 1,000 teams	10,000	1,000
References copied for 10,000 teams	100,000	10,000
References copied for 50,000 teams	500,000	50,000

The redesigned system is predicted to reduce sorting operations and copying by 90% for this workload.

E. Implement and Validate the Redesign (2 Marks)

15. Experimental Testing

The original and redesigned solutions are tested under the same conditions.

Three different team sizes are used:

- Experiment 1: 1,000 teams.
- Experiment 2: 10,000 teams.
- Experiment 3: 50,000 teams.

Each experiment requests the ranking 10 times without making any changes to the team list.

The following measurements are collected:

1. Execution time using System.nanoTime().
2. Number of sorting operations.
3. Number of team references copied.

Experimental Results

Record the actual execution times from your VS Code terminal.

Number of Teams	Original Time (ms)	Redesigned Time (ms)
1,000	Actual result	Actual result
10,000	Actual result	Actual result
50,000	Actual result	Actual result

Sorting Operation Comparison

Number of Teams	Original Sorts	Redesigned Sorts
1,000	10	1
10,000	10	1
50,000	10	1

Reference Copying Comparison

Number of Teams	Original	Redesigned
1,000	10,000	1,000
10,000	100,000	10,000
50,000	500,000	50,000

The redesigned solution reduces the number of complete sorting operations and list-copy operations.

The execution times may vary depending on the computer, Java version and system load. Therefore, the actual time values must be recorded from program execution.

Correctness Validation

Both implementations use the same Comparator.

The expected ranking is:

[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

The redesigned solution produces the same ranking as the original solution.

Therefore, the redesign improves repeated ranking requests without changing the ranking rules.

Complexity Comparison

Let n represent the number of teams and r represent the number of ranking requests without any changes to the data.

Original Solution:

- One ranking request: O(n log n).
- r ranking requests: O(r × n log n).
- Additional temporary space: O(n).

Redesigned Solution:

- Adding a team: O(1) amortized.
- First ranking request after modification: O(n log n).
- Subsequent ranking requests: O(1) to return the cached result.
- Total for r ranking requests: O(n log n + r).
- Additional retained cache space: O(n).

The redesigned solution is more efficient when ranking requests are frequent and team updates are less frequent.

F. Defend the Final Design (2 Marks)

16. Evaluation of the Redesigned System

The redesigned CachedLeaderboard successfully addresses the repeated sorting problem.

A. Was the Prediction Supported?

Yes. The redesigned implementation performs only one sorting operation for 10 consecutive ranking requests without any new team additions.

The original implementation performs 10 sorting operations under the same conditions.

Therefore, the reduction in repeated sorting is supported by the program's operation count.

B. Performance Improvement

The redesigned system reduces:

- Sorting operations from 10 to 1.
- List-copy operations from 10 to 1.
- Unnecessary object creation.
- Repeated processing of unchanged team data.

Actual execution-time improvement must be evaluated using the recorded benchmark results.

C. New Costs and Trade-offs

The redesigned system requires additional memory to store the cached ranking.

The cache must also be invalidated whenever team information changes.

When new teams are added, the next ranking request performs another complete sorting operation.

Therefore, the redesigned solution is most beneficial when ranking requests are more frequent than updates.

D. Situations Where the Original Solution Is Preferable

The original implementation may be preferable when:

1. Rankings are requested only once.
2. The number of teams is very small.
3. Teams are updated frequently.
4. Additional cache memory is not desirable.
5. A simple implementation is sufficient.

17. Final Recommendation

I recommend the CachedLeaderboard design for applications where ranking results are requested frequently but team information changes occasionally.

The redesign reduces repeated sorting and unnecessary list creation while preserving the original ranking order.

It also demonstrates that improving software performance does not always require replacing an existing collection. Identifying repeated work and avoiding unnecessary operations can provide a more effective solution.

18. Conclusion

The original Leaderboard implementation has a performance limitation because it copies and sorts all teams every time the ranking method is called. The redesigned CachedLeaderboard uses a caching mechanism to store and reuse the sorted result. Experimental comparison measures execution time, sorting operations and copied references. The redesigned solution reduces repeated work and improves performance for read-heavy applications, although it requires additional memory and cache invalidation when data changes.